Evolutionary search with a language-model mutator

Evolving solver programs through measured feedback

ENGR 689 · Texas A&M University · Fall 2026

How can we improve the algorithm that performs the search?

Optimization as search

When the best candidate is unknown, what can we do besides guess once?

Define a candidate space $\mathcal{A}$ and a measurable objective F. Seek

$$a^* \in \arg\max_{a \in \mathcal{A}} F(a).$$

Generate candidates $\rightarrow$ measure them $\rightarrow$ spend the next trial where the evidence suggests it may help.

Optimization is an evidence-guided search. An evolutionary algorithm adds a population, inherited variation and selection.

Population and fitness

Why keep several candidates instead of only the current champion?

A population stores alternative structures. Fitness measures each candidate under a declared evaluation protocol.

Illustrative scores: A $= 5$, B $= 7$, C $= 6$. Keeping B and C preserves two starting points for the next variation step.

A population supplies material for inheritance and exploration. Its size and selection rule determine what information survives.

Variation, evaluation and selection

What turns repeated candidate generation into an evolutionary algorithm?

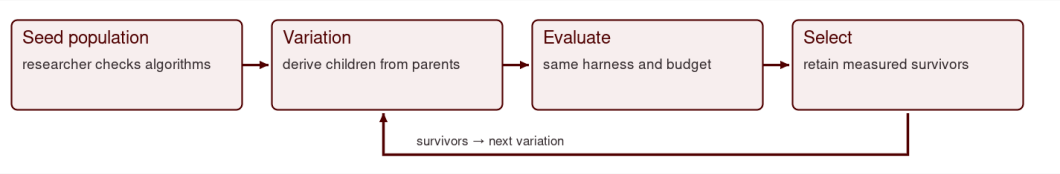


Figure 1: Seed once; derive children from survivors; evaluate and select again.

Mutation can perturb a parameter, replace a component or compose stages. Selection compares parents and children using the same fitness protocol.

Useful structure is inherited, varied and tested. Elitism retains measured parents when a mutation is worse.

Exploration and exploitation

Should the next mutation refine a strong candidate or try a different mechanism?

Exploit: adjust a promising parameter or refine an existing composition. **Explore:** replace a stage or try another algorithm family.

Too little variation can collapse the population. Too much unconstrained variation can waste evaluations on poor candidates.

Variation and selection must balance inheritance with discovery. Better starting material may help; a weak start does not imply permanent failure.

The language-model mutator

What changes if the variation operator can read programs and measured feedback?

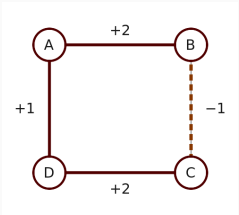
A large language model reads selected programs, trial outcomes and allowed edits, then proposes a motivated mutation.

For example: retain a vector-relaxation stage, then append local search to improve the rounded assignment. The proposal is a hypothesis.

The LLM supplies context-conditioned variation. The harness still supplies fitness. Its weights are not trained during this run.

Signed Max-Cut

How should vertices be partitioned when some edges prefer separation and others prefer agreement?



Positive edge: prefer different sides. Negative edge: prefer the same side.

$$C_G(s) = \sum_{(i,j)\in E} w_{ij} \frac{1 - s_i s_j}{2}, \quad s_i \in \{-1, +1\}.$$

Maximize the original signed cut. The comparison uses 1,000 vertices, 5,043 signed edges and reference 1098.

Local moves and feasible witnesses

Can the search move to a worse state while keeping a better result?

Flip one sign, $s_i \leftarrow -s_i$, with exact gain

$$\Delta_i = s_i \sum_{j:(i,j)\in E} w_{ij} s_j.$$

The exploratory assignment may worsen. A separate best-so-far copy preserves the strongest valid witness seen during that execution.

Separate the moving state, best witness and strategy fitness. Only a complete evaluation makes a strategy eligible for ranking.

Solver programs

What can Luna edit if the assignment itself is not the evolving artifact?

A program has one to three stages: algorithm, configuration, work cap and start rule.

```
bm(max_sweeps=20, rounding_count=8); retained
→ bls(max_work=9000); retained
```

Execute a program → obtain signs → recompute the original cut → summarize three trials → rank the program.

Evolve the solver; evaluate its witness. Trusted kernels execute the stages. Unchecked algorithms remain available to the mutator.

Four numerical algorithms

Which mechanism changes when we choose a different solver stage?

Tabu Search: best admissible flips, temporary reversal bans and aspiration.
Simulated Annealing: temperature-dependent acceptance of random flips.

Breakout Local Search: improving descent followed by adaptive perturbations.
Rank-two Burer–Monteiro: unit-vector refinement, projection to signs and polish.

Acceptance, memory, escape and representation are different design levers. Full examples and parameter meanings are available in the algorithm guide.

Initial population and fair starts

Do the solver checkboxes change the initial population or the whole search space?

Checkboxes choose P_0. Every selected program and proposed child receives copies of the same raw starts for seeds 3, 4 and 5.

All four trusted kernels remain available to Luna. Both current graphs allow 8 numerical seconds per trial. The selected seed programs produce distinct initial fitness on the 1,000-vertex graph.

Change initialization deliberately and hold the comparison protocol fixed. A missing seed algorithm can still appear in a later mutation.

Designing the three experiments

Which differences can our initialization comparison actually explain?

Demos 1 and 2: same signed graph, seeds, trial allowance, mutation catalog and call cap. Change only the initial solver suite.

Report initial median, calls and time to the median target, all-trial robustness, numerical effort and total teaching-run time.

Declare the success criterion before comparing runs. The third graph is a separate dense real-weight stress case.

Demo 1: a limited initialization

Can local-search seeds recover a useful relaxation method through mutation?

Use the 1,000-vertex signed graph, seed 3, **Tabu Search and Simulated Annealing**. The initial best median is 975; reference 1098 is unproven.

All four kernels remain available. Inspect parent programs, the actual request, public proposals and complete measured child fitness.

A limited suite defines a starting point, not a permanent restriction. Follow improvement, reference gap, numerical effort and model calls.

Demo 2: a researcher-selected suite

What changes when measured domain knowledge supplies stronger starting programs?

Keep graph, seeds and caps fixed. Select **Tabu Search, Breakout Local Search and rank-two Burer–Monteiro**.

The initial best median is 1012, compared with 975 for the limited suite. The reference gap falls from 123 to 86 before any model call.

Researcher knowledge improves this starting population. Evaluate later gains and costs; a stronger start does not guarantee a faster run.

Comparing the evidence

If every method eventually reaches the same target, how should we compare them?

Compare initial fitness, complete-program improvement, reference gap and calls or time to a declared target. Retain unsuccessful and censored proposals.

Initial medians: **921 / 975 / 992 / 1012**. These distinguish the four configured solvers on the replacement graph; reference **1098** is unproven.

A common final score can hide different search costs. An unreached reference must remain visibly unreached.

Demo 3: a dense real-weight stress case

Will the same architecture work when interactions are dense and weights are real-valued?

Use **100 vertices, 4950 real-valued signed edges**, all four initial solvers and supplied reference **402.5209**, unproven.

Eight numerical seconds per trial; three Luna calls maximum. Inspect composition, rejection, completeness and the actual surviving programs.

Evaluate the dense case on its own evidence. A reference match is a feasible result; it is not a proof of optimality or broad model superiority.

Reading solver evidence

Which record proves that a solver improved: the graph, the hypothesis or the completed trials?

Require original-order integer signs and a matching graph hash. Recompute the objective from the original signed edges with two scoring paths.

Rank complete programs by median, minimum, fewer stages and stable genome ID. Retain a valid best witness separately, including one from a censored trial.

Selection follows complete measured evidence. Preserve failures, timeouts and plateaus so students can inspect the full mechanism.

The agentic evolutionary framework

Can we now name the five choices we implemented?

$(\mathcal{A}, f, \mu, \sigma, P_0)$: artifact space, fitness, mutation, selection and initial population. Checkboxes choose P_0; they do not change $\mathcal{A}$.

$$Q_t \sim \mu(\cdot \mid P_t, H_t, B_t), \quad P_{t+1} = \sigma(P_t \cup E(Q_t)).$$

H_t: measured history. B_t: remaining budget. E: validation and complete evaluation.

The LLM is the mutation operator inside an externally measured evolutionary loop. For the course minimization convention, $f(a) = -\operatorname{median}_j C_G(A_a(G, z_j))$.

Applying the framework to research

Which component would be hardest to supply honestly for your own research problem?

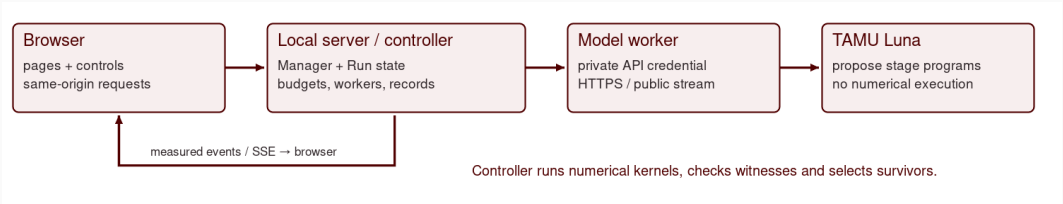
Specify the artifact, automatic evaluator, mutation context, survivor rule and initial population. Declare budgets and a success criterion before running.

Use domain knowledge, compare strong numerical baselines, and test unseen instances before claiming generalization.

Build trustworthy measurement and inspectable data flow first. Then decide where a language-model mutator earns its cost.

Appendix A1: Server architecture

What coordinates the browser, numerical search and model?



One loopback server owns run state and workers, serves every teaching page, and streams events to the browser.

The server orchestrates; workers execute; events explain. The API key stays in the server-side client.

Appendix A2: The API request and context

What is actually sent when we press “Send parents to Luna”?

```
model: protected.gpt-5.6-luna
messages:
  system: instructions + response schema
  user: serialized round context
stream: true; tools: []
```

Context: up to three genomes and trial evidence; failures and tried IDs; algorithm defaults, bounds and remaining budget. The worker adds HTTPS authorization separately.

The context carries measured search memory. Solution vectors, reference answers and the API key are excluded from the model’s context.

Appendix A3: Response and loop closure

When does streamed text become a candidate that can survive?

Public chunks → completed JSON → known parent IDs → schema, bounds and duplicate checks → admitted stage program.

Owned numerical trials → independent original-edge scores → complete-candidate ranking → new survivors and failure feedback for the next request.

Text is a proposal until validated and measured. Server-sent events expose progress; selection supplies the next mutation context.